

Description	Keystroke		Function	Note
Comments	Perl comments start with #. See ℹ Comments for the generic commands that manage and control comments			
Toggle display of comments in buffer or active region	<f11> ; ;	(hide/show-comments-toggle &optional START END)	Toggle hiding/showing of comments in the active region or whole buffer. If the region is active then toggle in the region. Otherwise, in the whole buffer.	
	 This requires the hide-comnt.el package (see ℹ Comments).  PEL activates it when the pel-use-hide-comnt user option is t .			
Generic code skeletons tempo skeletons See also: ℹ Inserting Text T Templates	Several mechanisms have been developed to allow easy insertion of predefined text in Emacs.  PEL does not yet define skeletons for Perl. You can use the generic one. - Emacs provides the built-in skeleton mechanism and the tempo skeletons. - PEL supports both. They are used a little bit differently. PEL provides generic tempo skeletons you can use for Perl until PEL adds Perl-specific skeletons. - PEL provides key bindings to the tempo skeletons: the generic code templates, accessible via the <f6> prefix key, and the language-specific code templates, accessible via the <f12> key prefix.			
ℹ Customize PEL Text Insertions control for Perl code skeletons.	<f6> <f2>	(pel-customize-pel &optional OTHER-WINDOW)	Open the customization groups that control the format of the various skeletons including the generic skeleton used by the <f6> h key and the <f12><f12> h key (see below). If OTHER-WINDOW is non-nil (use C-u), display in other window.	
	<f12> <f12> <f2>	(pel-customize-generic-skels &optional OTHER-WINDOW)		
Insert generic file module header block — Language agnostic	<f6> h	(pel-generic-file-header)	Insert a file header block at the top of the file. Works only for buffer visiting a file.  The command key binding <f6> h is available only 1 second after Emacs has started.  As mentioned above PEL does not yet define Perl-specific skeletons, this uses the generic one.	
	<f12> <f12> h			
After inserting the template, navigate though areas that must be filled with: - tempo-forward-mark: C-c . - tempo-backward-mark: C-c ,	 Specify the format of the header via the user-options in the pel-pkg-generic-code-style customization group accessible via <f6> <f2> Inside a Perl buffer, <f12> <f2> provides access to the following customization groups:  After inserting a template, use tempo-forward-mark and tempo-backward-mark to move to the beginning of each section that must be filled.			
Toggle pel-tempo-mode	<f6> SPC	(pel-tempo-mode &optional ARG)	Toggle PEL tempo mode on/off.	
	<f12> <f12> SPC			
PEL tempo mode activates C-c . and C-c , as well as to C-c C- . and C-c C- , key bindings to navigate across tempo mark hot-spots. When pel-tempo-mode is active the pel-tempo-mode lighter (⚡) is shown on the status bar. The second set of keys are only available in graphics mode.  The pel-generic-file-header command inserts the text using a tempo skeleton: the PEL tempo mode is automatically activated by typing <f6> h .				
Expand any tag in template	<f6> <f12>	(tempo-complete-tag &optional SILENT)	Look for a tag and expand it. All the tags in the tag lists in 'tempo-local-tags' (this includes 'tempo-tags') are searched for a match for the text before the point. The way the string to match for is determined can be altered with the variable 'tempo-match-finder'. If 'tempo-match-finder' returns nil, then the results are the same as no match at all. - If a single match is found, the corresponding template is expanded in place of the matching string. - If a partial completion or no match at all is found, and SILENT is non-nil, the function will give a signal. - If a partial completion is found and 'tempo-show-completion-buffer' is non-nil, a buffer containing possible completions is displayed.	
Note: PEL default skeleton does not use tags.	<f12> <f12> <f12>			
Navigation Commands	Some navigation commands have special behaviour for Perl; they are shown below. They are available in perl-mode and cperl-mode . There's also several generic navigation commands that work in Perl buffer too. See ℹ Navigation for those.			
Move to beginning of function	C-M-a <f12> <up>	(perl-beginning-of-function &optional ARG)	Move backward to next beginning-of-function, or as far as possible. With argument, repeat that many times; negative args move forward.	
To end of function	C-M-e <f12> <down>	(perl-end-of-function &optional ARG)	Move forward to next end-of-function. - The end of a function is found by moving forward from the beginning of one. - With argument, repeat that many times; negative args move backward.	
Move to next interpolated	C-c C-v	(cperl-next-interpolated-REx &optional SKIP BEG LIMIT)	Move point to next REx which has interpolated parts. - SKIP is a list of possible types to skip, BEG and LIMIT are the starting point and the limit of search (default to point and end of buffer). - SKIP may be a number, then it behaves as list of numbers up to SKIP; this semantic may be used as a numeric argument. - Types are 0 for / \$rex /o (interpolated once), 1 for /\$rex/ (if \$rex is a result of qr//, this is not a performance hit), t for the rest	
	C-c C-x	(cperl-next-interpolated-REx-0)	Move point to next REx which has interpolated parts without //o.	
	C-c C-y	(cperl-next-interpolated-REx-1)	Move point to next REx which has interpolated parts without //o. - Skips REXes consisting of one interpolated variable. - Note that skipped REXen are not performance hits.	
Marking Commands	The following marking commands are specialized for Perl. See ℹ Marking for the generic ones that can also be used in Perl buffers.			
Mark function	C-M-h	(perl-mark-function)	Put mark at end of Perl function, point at beginning	
Indentation Control	The following indentation control commands are specialized for Perl. See ℹ Indentation for the generic ones that can also be used in Perl. The indentation behaviour of the <tab> key is controlled by the perl-mode and cperl-mode customization user-options. Use <f12> <f3> to open customization.			
Show currently used indentation style	<f12> <f4> s	(pel-perl-show-style)	Show the name of the currently used indentation style.	
Select new/restore old indentation style	<f12> <f4> <TAB>	(pel-perl-set-style)	Set Perl indentation style to named style. Prompt for indentation style name and apply it.  This change does not persist. Manually access the customized buffer and save it.	
Indent perl-mode ➡	<tab>	(perl-indent-command &optional ARG)	Indent Perl code in the active region or current line. In Transient Mark mode, when the region is active, reindent the region. - Otherwise with a prefix argument, reindent the current line unconditionally. - Otherwise if 'perl-tab-always-indent' is nil and point is not in the indentation area at the beginning of the line, insert a tab. - Otherwise, indent the current line. If point was within the indentation area, it is moved to the end of the indentation area. If the line was already indented properly and point was not within the indentation area, and if 'perl-tab-to-comment' is non-nil (the default), then do the first possible action from the following list: - delete an empty comment - move forward to start of comment, indenting if necessary - move forward to end of line - create an empty comment - move backward to start of comment, indenting if necessary.	
		(cperl-indent-command &optional WHOLE-EXP)	Indent current line as Perl code, or in some cases insert a tab character. - If 'cperl-tab-always-indent' is non-nil (the default), always indent current line. Otherwise, indent the current line only if point is at the left margin or in the line's indentation; otherwise insert a tab. - A numeric argument, regardless of its value, means indent rigidly all the lines of the expression starting after point so that this line becomes properly indented. The relative indentation among the lines of the expression are preserved.	
Indent continued expression cperl-mode ➡	C-M-q	(perl-indent-exp)	Indent each line of the Perl grouping following point.	
		(cperl-indent-exp)	Simple variant of indentation of continued-sexp. - Won't indent comment if it starts at 'comment-indent' or looks like continuation of the comment on the previous line. - If 'cperl-indent-region-fix-constructs', will improve spacing on conditional/loop constructs.	
Indent region	C-M-\	(cperl-indent-region START END)	Indents all the lines whose first character is between START and END inclusive. - Won't indent comment if it starts at 'comment-indent' or looks like continuation of the comment on the previous line. - If 'cperl-indent-region-fix-constructs', will improve spacing on conditional/loop constructs.	
Newline and indent cperl-mode ➡	C-j	(newline-and-indent)	Insert a newline at point, then indent the newly created line. Use it to split a line. Indentation is done using the value of 'indent-line-function' which is set to cperl-indent-line.	
Insert Perl new line	C-c C-j	(cperl-linefeed)	Go to end of line, open a new line and indent appropriately. If in POD, insert appropriate lines.	
	It's a convenience replacement for typing return . It puts point in the next line with proper indentation, or if you type it inside the inline block of control construct, like <pre>foreach (@lines) {print; print}</pre> and you are on a boundary of a statement inside braces, it will transform the construct into a multiline and will place you into an appropriately indented blank line. Use C-j for usual 'newline-and-indent' behavior. See 'cperl-electric-linefeed' documentation.			

Description	Keystroke	Function	Note
Insert matching parens toggle with C-c C-e	(< [{	(cperl-electric-paren ARG)	Insert an opening parenthesis or a matching pair of parentheses. Controlled by 'cperl-electric-parens' and 'cperl-hairy' user-options.
	)]	(cperl-electric-rparen ARG)	Insert a matching pair of parentheses if marking is active. - If not, or if we are not at the end of marking range, would self-insert. - Controlled by 'cperl-electric-parens' and 'cperl-hairy' user-options.
Insert : and indent	:	(cperl-electric-terminator ARG)	Insert character and correct line's indentation.
Insert ; and indent	;	(cperl-electric-semi ARG)	Insert character and correct line's indentation.
Insert { and indent	{	(cperl-electric-lbrace ARG &optional END)	Insert character, correct line's indentation, correct quoting by space.
Insert } and indent	}	(cperl-electric-brace ARG &optional ONLY-BEFORE)	Insert character and correct line's indentation. If ONLY-BEFORE and 'cperl-auto-newline', will insert newline before the place (even in empty line), but not after. If after ")" and the inserted char is "{", insert extra newline before only if 'cperl-extra-newline-before-brace'.
Deleted and possibly untabify	DEL	(cperl-electric-backspace ARG)	Backspace, or remove whitespace around the point inserted by an electric key. Will untabify if 'cperl-electric-backspace-untabify' is non-nil.
cperl-mode	PEL supports 2 cperl-mode implementations: Emacs' cperl-mode and  HaraldJoerg/cperl-mode (more complete) activated by the pel-use-perl user-option. The following cperl commands toggle some of its electric behaviour.		
toggle Perl auto-help	C-c C-h a	(cperl-toggle-autohelp)	Toggle the state of Auto-Help on Perl constructs (put in the message area). Delay of auto-help controlled by 'cperl-lazy-help-time':  By default it is nil, which (like null) prevents activation of the auto-help. To activate it, set cperl-lazy-help-time to an integer value.
Toggle auto-newline	C-c C-a	(cperl-toggle-auto-newline)	Toggle the state of 'cperl-auto-newline'.
Toggle electric mode	C-c C-e	(cperl-toggle-electric)	Toggle the state of parentheses doubling in CPerl mode. When typing an opening parens character the closing one is automatically entered.
Toggle auto-fill mode	C-c C-f	(auto-fill-mode &optional ARG)	Toggle automatic line breaking (Auto Fill mode). - With a prefix argument, enable Auto Fill mode if the prefix argument is positive, disable it otherwise. - When Auto Fill mode is enabled, inserting a space at a column beyond 'current-fill-column' automatically breaks the line at a previous space.
	<f11> t f C <f11> RET		
Toggle keyword expansion	C-c C-k	(cperl-toggle-abbrev)	Toggle the state of automatic keyword expansion in CPerl mode.
Toggle space fix	C-c C-w	(cperl-toggle-construct-fix)	Toggle whether 'indent-region'/'indent-sexp' fix whitespace too.
here-doc support	The following cperl-mode commands operate on Perl here documents .		
Narrows to here-doc	C-c C-n	(cperl-narrow-to-here-doc &optional POS)	Narrows editing region to the HERE-DOC at POS. POS defaults to the point.
Spell-check here-docs	C-c C-d	(cperl-here-doc-spell)	Spell-check HERE-documents in the Perl buffer. If a region is highlighted, restricts to the region.
Spell-check POD documentation	C-c C-p	(cperl-pod-spell &optional DO-HERES)	Spell-check POD documentation. - If invoked with prefix argument, will do HERE-DOCs instead. - If a region is highlighted, restricts to the region.
Verify & refactor Perl code	The following commands provide Perl verification and refactoring facilities.		
perltidy  currently requires a lperltidy to be present on local host event when processing aremote file.	The following 2 commands run perltidy between areas of the Perl code in the current buffer and the tidied version of that code. All work is done inside Emacs buffers, no file is affected. After generating the tidied code the functions open an ediff session to compare your code and the tidied code, allowing you to decide what to use. Requires  HaraldJoerg/cperl-mode activated by the pel-use-perl user-option.  The perl-tidy-ediff commands execute the perltidy identified by perl-tidy-command user-option with options identified by the perl-tidy-ediff-args user-option. See Perl::Tidy @metacpan for the perltidy list of options, also Online PerlTidy , which provides help on the various options categorized by feature set.		
Run perltidy on current buffer or region. Ediff changes.	<f12> T	(pel-perl-tidy-ediff)	Run perltidy on current buffer, than start an ediff session, comparing the original source with perltidy output. Error messages are saved in the "perl-tidy-errors" buffer. If an area is marked, run perltidy on the marked area only.
Run perltidy on current subroutine. Ediff changes. See ℳ Narrowing	<f12> t	(perl-tidy-ediff-sub)	Run the perltidy program on the subroutine that stats before point and start en ediff session to compare the original code with the tidied version. - Error messages are saved in the "perl-tidy-errors" buffer.  The buffer is automatically narrowed to the current function. - When quitting the ediff session it remains narrowed. Use C-x n w to widen the buffer back.
perlcritic	Perform static code analysis with perlcritic (which must be installed separately).		
Check code with perlcritic  ℳ Tramp -aware : run remote host's perlcritic on remote file.	<f12> c	(pel-perl-critic &optional VERBOSE)	Validate the Perl file visited in current buffer with perlcritic . Report error if it's not installed. - With optional VERBOSE prefix argument, print extra information: - Full name of the Policy module that created the violation - Full diagnostic discussion of each Perl Best Practice (PBP) violation. Show errors in compilation-mode buffer in a format that allows navigation.
Other refactoring			
Find/fix missing whitespace code	C-c C-b	(cperl-find-bad-style)	Find places in the buffer where insertion of a whitespace may help. Prompts user for insertion of spaces. Currently it is tuned to C and Perl syntax.
Refactor: if (A) {B}' → 'B if A;'	C-c C-t	(cperl-invert-if-unless)	Change 'if (A) {B}' into 'B if A;' etc (or visa versa) if possible. If the cursor is not on the leading keyword of the BLOCK flavor of construct, will assume it is the STATEMENT flavor, so will try to find the appropriate statement modifier.
Lining up Perl Code : ℳ Align	It's often best to line up Perl code vertically , arranging elements as tables, to help reader understand the code intent visually. This technique is also promoted in Damian Conway's Perl Best Practice book, Vertical Alignment section of chapter 2. See pel-align-words-vertically in ℳ Align.		
Lineup	- C-M- <f12>	(cperl-lineup BEG END &optional STEP MINSHIFT)	Lineup construction in a region. - Beginning of region should be at the start of a construction. - All first occurrences of this construction in the lines that are partially contained in the region are lined up at the same column. - MINSHIFT is the minimal amount of space to insert before the construction. - STEP is the tabwidth to position constructions. - If STEP is nil, 'cperl-lineup-step' will be used (or 'cperl-indent-level', if 'cperl-lineup-step' is nil). - Will not move the position at the start to the left.
Testing Perl code	The following command provides a way to test Perl code locally. See Perl 5 Syntax for web-sites that provide a Perl interpreter.		
Start Perl REPL See: ℳ start Shells/REPLs	<f11> z r P <f12> z	(perl-repl)	Run a Perl REPL in a "Perl-REPL" buffer.  Requires the perl-repl external package  activated by perl-use-perl-repl user-option. - The perl-repl-file-path user option specifies the name of the Perl REPL program, which may optionally specify the explicit file path. - PEL provides the perl-repl shell script which uses the Perl command line.
Perl Live Coding ★★	 perl-live-coding external package supported  when pel-use-pel-live-coding user-option is set. The "perl-live" buffer provides a much more powerful Perl interpreter.		
Run perl code in *perl-live*	<f12> 1	(pel-perl-live-run)	Start or open an existing perl-live session buffer. Invokes perl-live-run and open the "perl-live" buffer automatically, a comint mode buffer.  The <f12> 1 key binding is always bound in a perl-mode buffer. This is not the case for the following key bindings that become accessible once perl-live-run was first executed (via this command or via M-x perl-live-run)
The *perl-live* buffer operates in comint-mode which provides a set of commands to navigate across lines and history, as well as other. Type <f1> m inside the "perl-live" buffer to see these commands.	C-c C-1	(perl-live-run)	
Stop	C-c C-p	(perl-live-stop)	Stop perl-live session. The "perl-live" buffer gets disconnected from the Perl process. It can be restarted (without closing the buffer) with a new execution of perl-live-run.
Evaluate Perl code line/region	C-c C-c	(perl-live-eval-region-or-line)	Evaluate a line or region of Perl code. When evaluating a single line, point moves to the next line.
Evaluate Perl code in block	C-M-x	(perl-live-eval-sexp)	Evaluate Perl code between braces

Emacs & Perl — References

Document	Notes
Also see: Perl 5 Syntax	
Perl @ Wikipedia	
perl.org	
Learn Perl @ perl.org • Perl Style Guide	• Perl Tutorial - a gentle introduction to Perl with several examples over a browsable set of pages. • Perl Intro - a quick introduction to Perl • Online Perl books • Beginning Perl
Perl Reference Manuals	• Perl Keywords. • Perl Operators. Also see the Perl ABC Operator page: organizes the information in sections (but has some markup typos). • Perl Functions
Is Perl still relevant? Most probably.	• What makes Perl relevant in 2022? @ Stackoverflow blog. • Perl is dying quick. Could be extinct by 2023. The HFT Guy, 2019. (which includes several invalid points). • My point is that Perl was popular, there's a lot of Perl code still being used and it's worth knowing and being able to write and edit Perl code (which was certainly not the first programming language as state by the article). But anyway, the post represents a point of view (and has many people commenting on it). • Perl is making a comeback: 5 reasons why it's worth learning. Posted January 6, 2023 by Lucas Rees. • Why Perl Didn't Win on outspeaking.com, updated December 21, 2020
Perl and Secure Coding Practice and Tools	• Security Issues in Perl Scripts - by Jordan Dimov - Discussion of misused and overlooked features of Perl from the security point of view. • SEI CERT Perl Coding Standard @ Carnegie Mellon University • The CERT Perl Secure Coding Standard, by David Svoboda , June 25, 2012 • perlsec describes Perl security. Tools: • The perlritic script uses Perl::Critic to scan Perl code and provides some listing advices. • On some Linux distros you can install it with: <code>sudo dnf install perl-Perl-Critic</code> • The perltidy application reformats Perl code to the promoted format. • The zarn static analyzer, hosted on Github, is another static analyzer for Perl. (Quite immature as of Oct. 2024).
Perl File name extensions	• .pl Perl non executable libraries, also used for Perl scripts (but a file with no extension and a shebang line is also fine, and preferable, allowing the invocation of the script without having to type the '.pl'). • .pm Perl modules. • .plx Used by Active State implementation. Identifies executable Perl scripts. Also used elsewhere to distinguish from Prolog (which also uses the .pl file extension). • .pls • .xs • .t • .pod Plain Old Documentation files, a lightweight markup language used to document Perl code. • See also perlpod. • .ph • .cgi • .psgi
Perl programs	• Perl command line options
• perl	Perl language interpreter
• perlbug / perlthanks	Describes how to submit bug report on Perl
• perldoc	Print Perl Documentation, looking it up in the .pod format embedded in perl installation. Support following options: • -f : built-in functions • -q : FAQ keyword search • -v : variable • -a perl API
• perlivp	• Perl Installation Verification Procedure : checks Perl installation. • Part of perl-level package.
• perltidy	• PerlTidy @ Wikipedia , PerlTidy Home Pages: @SourceForge @GitHub. Perl::Tidy GitHub repo, Perl::Tidy @meta::cpan • Online PerlTidy , which provides help on the various options categorized by feature set.
perlsec - Perl security	
Perl Community	Perl has a long history and is quite vast. Here's a collection of links to various web sites that can provide information about it. It is far from complete and collected without much background on what happened in lots of cases and no opinion yet taken on most of this. • strictures vs Schmorp common::sense, Marc A.Lehman common::sense package.